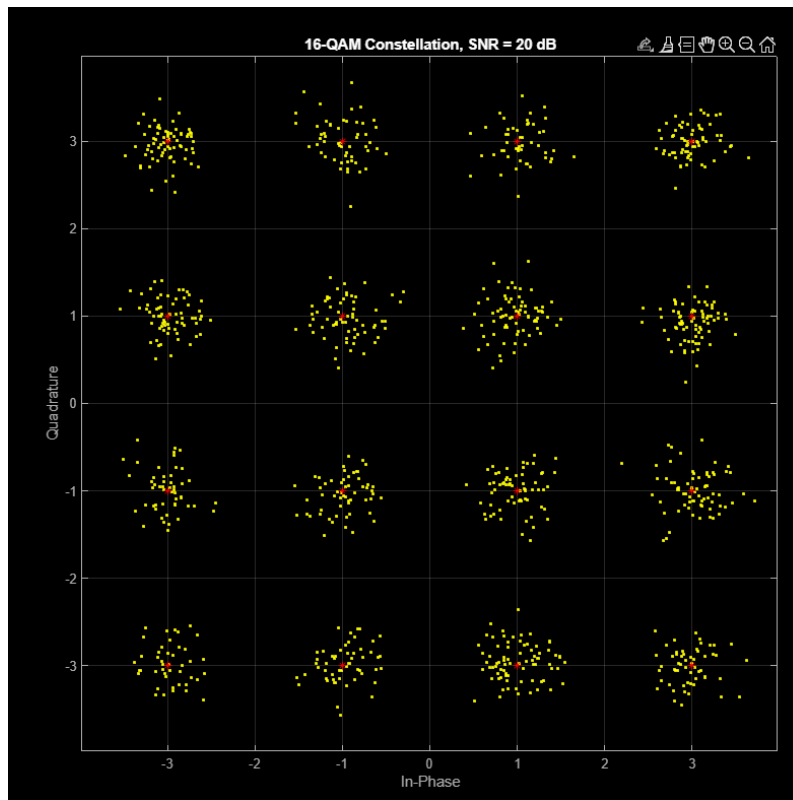


Lab 10: Quadrature Amplitude Modulation

Mariana Rivera, Sarah Boyd, Fatima Saldaña-Gutierrez

April 7, 2026



OBJECTIVES

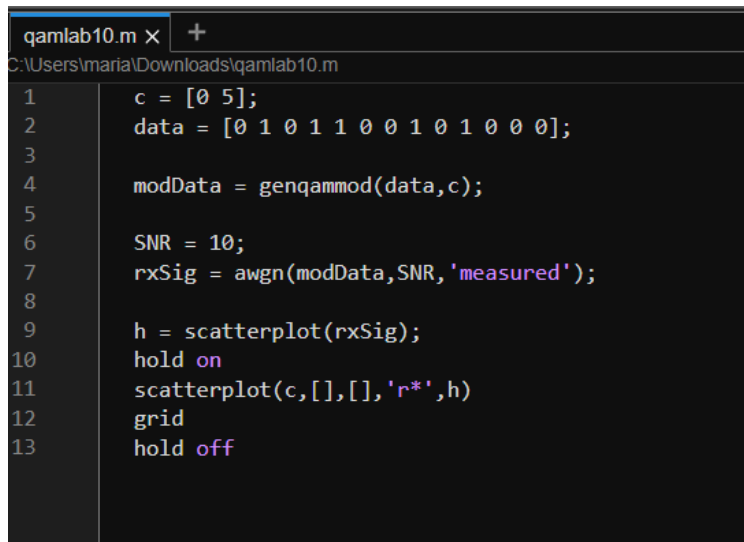
- Learn how to interpret a constellation diagram for QAM
- Examine how choice of modulation scheme affects Bit Error Rate (BER) and throughput
- Investigate effect of signal to noise ratio on Bit Error Rate (BER)
- Explore the effect of transmitting power on BER
- Learn to use MATLAB Communications Toolbox

INTRODUCTION

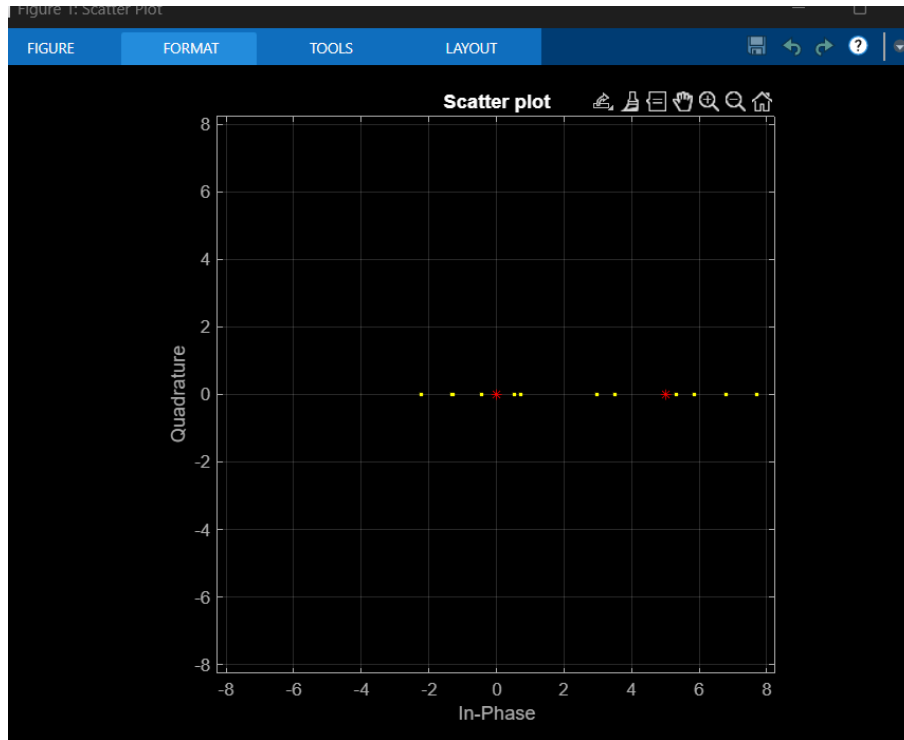
Quadrature Amplitude Modulation (QAM) is a digital modulation technique that uses both amplitude and phase changes to transmit information. Unlike ASK that only changes the amplitude of one carrier or PSK which changes the phase of the carrier, QAM uses two carrier components. Those carrier components are the in-phase component, I, and the quadrature component, Q. The components are 90 degrees out of phase, allowing data symbols to be represented as points on a two-dimensional constellation diagram.

The purpose of this lab was to examine how different modulation schemes behave in the presence of noise. MATLAB was used to create and analyze the following constellations: 2-ASK, 16-ASK, 16-QAM, and 16-PSK. Random data symbols were modulated, passed through a white Gaussian noise channel and then demodulated to measure symbol error rate and bit error rate. We also reviewed the effects of signal to noise ratio (SNR) and transmit power on constellation points and how it affected their spacing. Through these simulations, the relationship between constellation spacing, transmit power, SNR, and error were analyzed.

1-4. Creating a 2-ASK constellation with arrays:

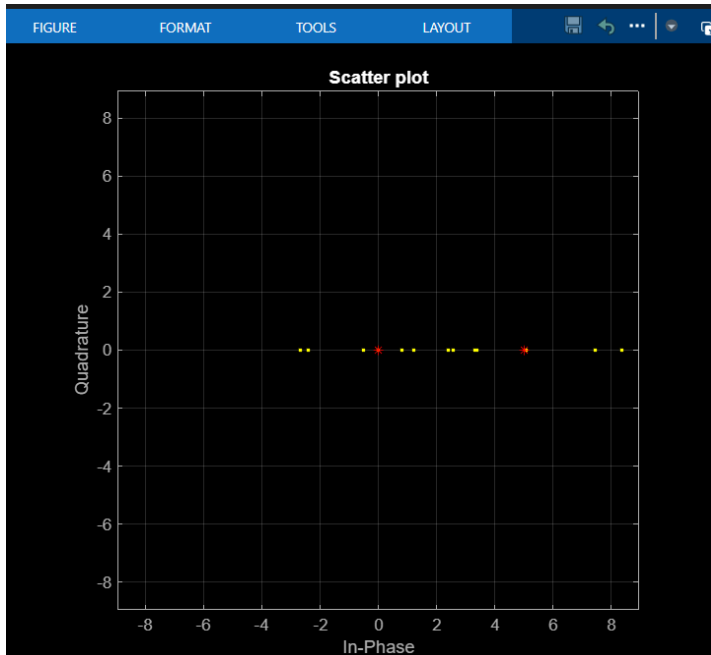
A screenshot of a MATLAB script editor window titled 'qamlab10.m'. The script contains 13 lines of code for creating a 2-ASK constellation. Line 1: 'c = [0 5];' Line 2: 'data = [0 1 0 1 1 0 0 1 0 1 0 0 0];' Line 3: (empty) Line 4: 'modData = genqammod(data,c);' Line 5: (empty) Line 6: 'SNR = 10;' Line 7: 'rxSig = awgn(modData,SNR,'measured');' Line 8: (empty) Line 9: 'h = scatterplot(rxSig);' Line 10: 'hold on' Line 11: 'scatterplot(c,[],[],'r*',h)' Line 12: 'grid' Line 13: 'hold off'

```
1 c = [0 5];
2 data = [0 1 0 1 1 0 0 1 0 1 0 0 0];
3
4 modData = genqammod(data,c);
5
6 SNR = 10;
7 rxSig = awgn(modData,SNR,'measured');
8
9 h = scatterplot(rxSig);
10 hold on
11 scatterplot(c,[],[],'r*',h)
12 grid
13 hold off
```

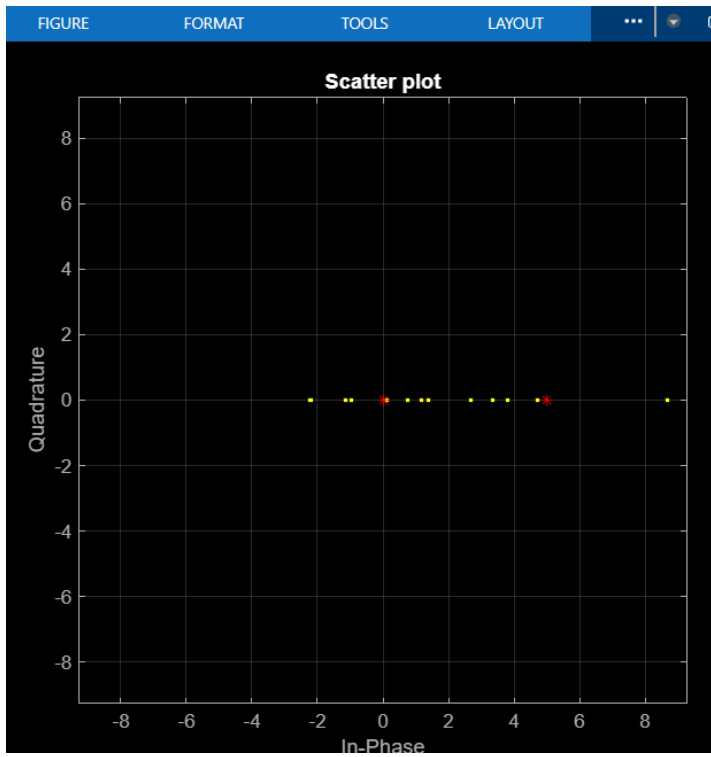


5. Experiment with the SNR value, by rerunning the script for SNR = 3, 6, 10, 20, 30, 40. What happens to the constellation of the received symbols once we increase the SNR? Capture screenshots to support your claims

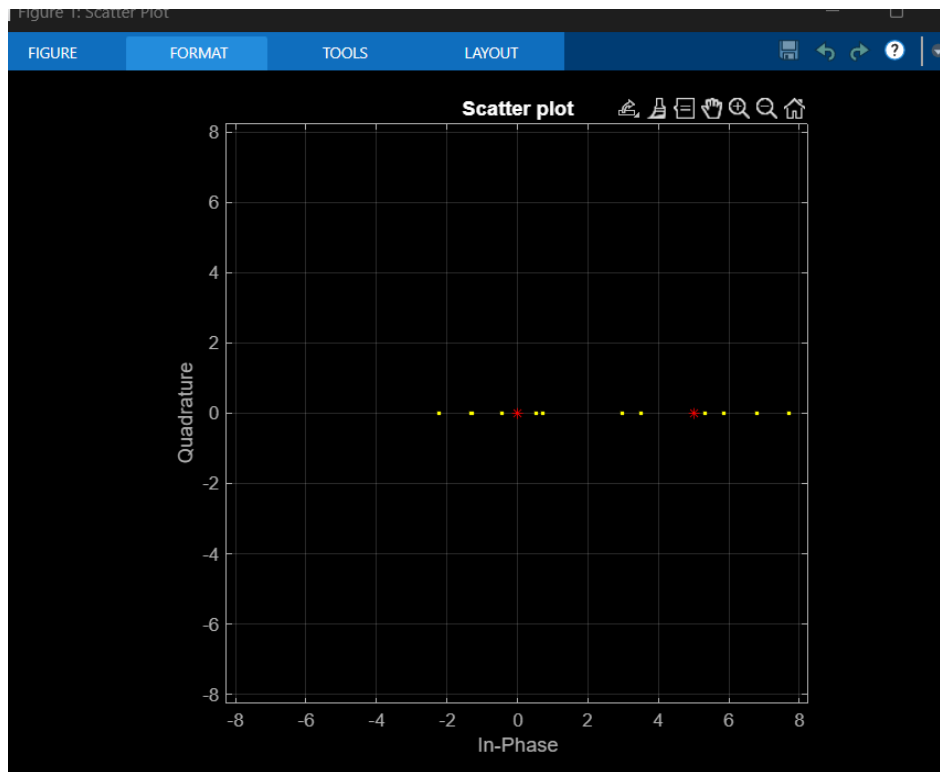
SNR = 3



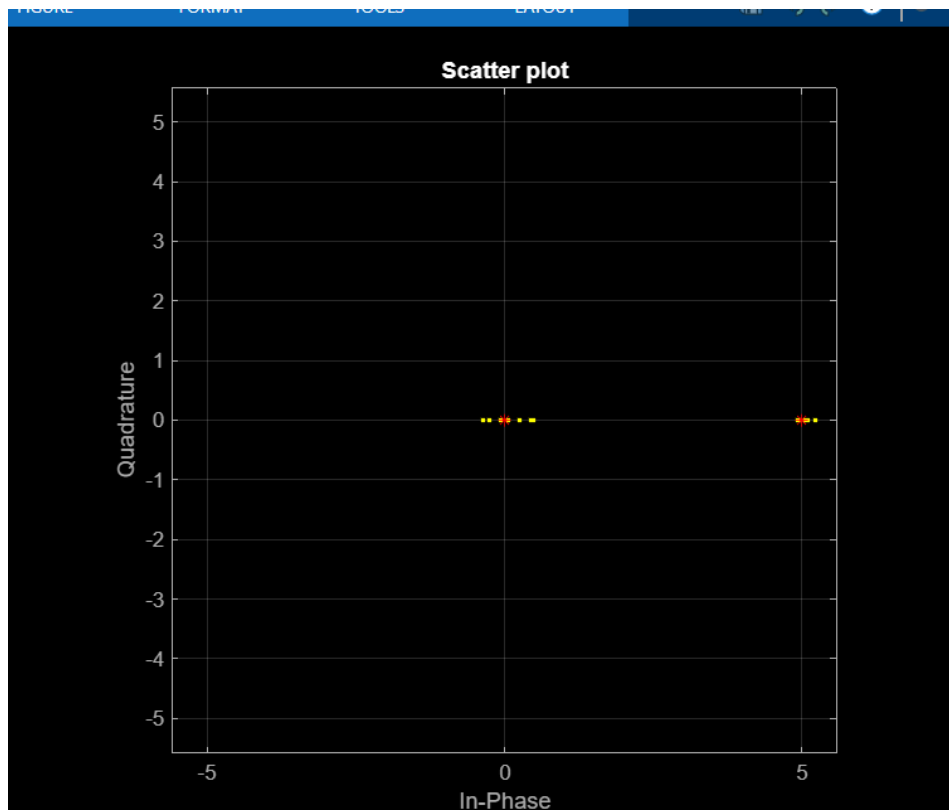
SNR = 6



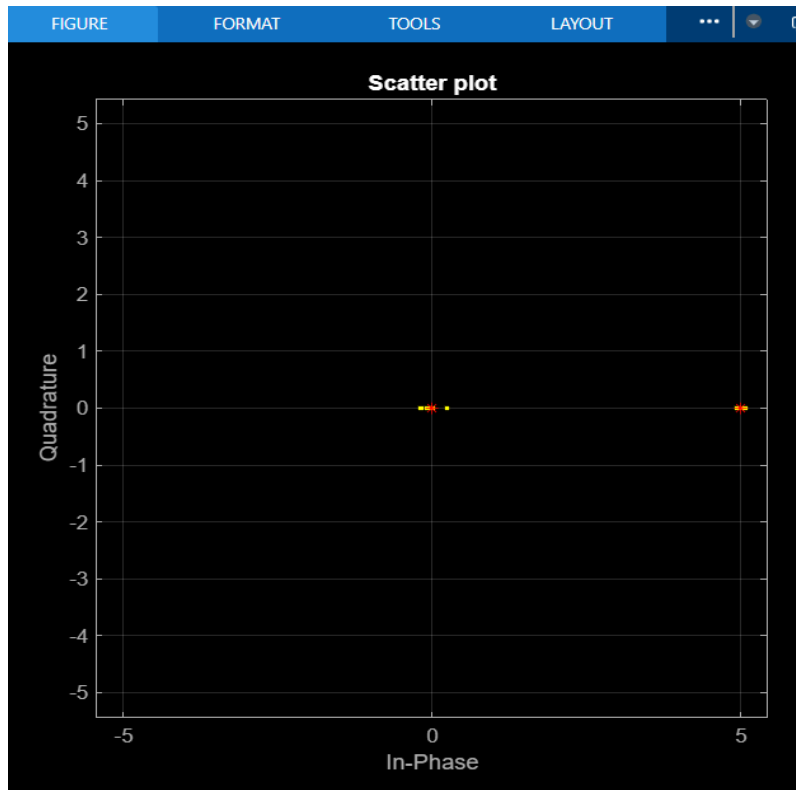
SNR = 10



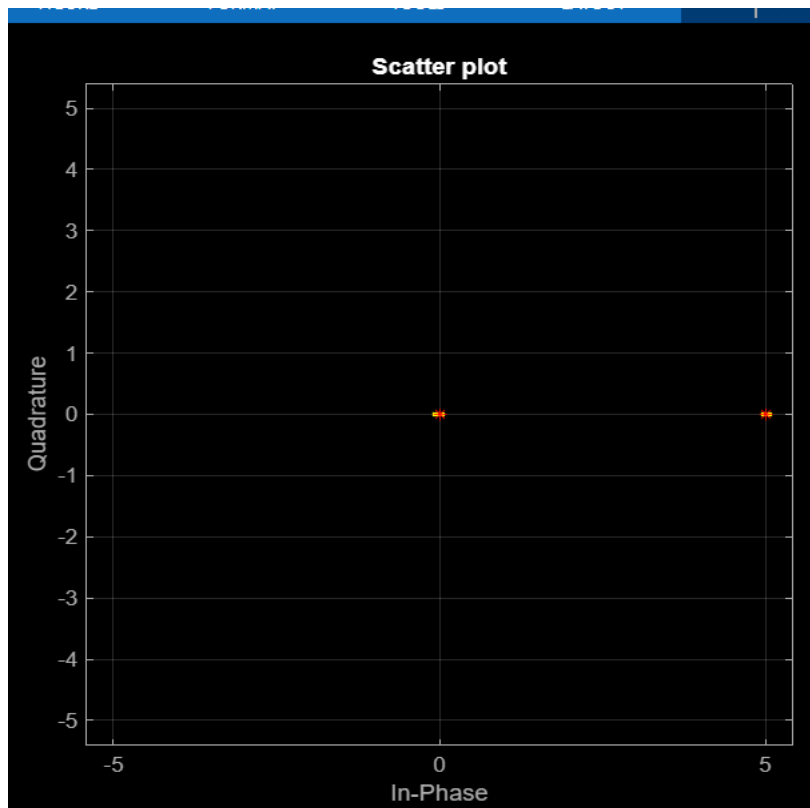
SNR = 20



SNR = 30



SNR = 40



What happens to the constellation of the received symbols once we increase the SNR?

As SNR increases, the constellation points cluster more tightly around the ideal red constellation points.

6. Empty in the lab.

7. Demodulate the data using `genqamdemod()`. Compare the original data with the demodulated data to check if there were any errors.

Record the constellation plot and error information for this run with $c = [0 \ 5]$ and $\text{SNR} = 10 \text{ dB}$.

```
c = [0 5];
data = [0 1 0 1 1 0 0 1 0 1 0 0 0];

modData = genqammod(data,c);

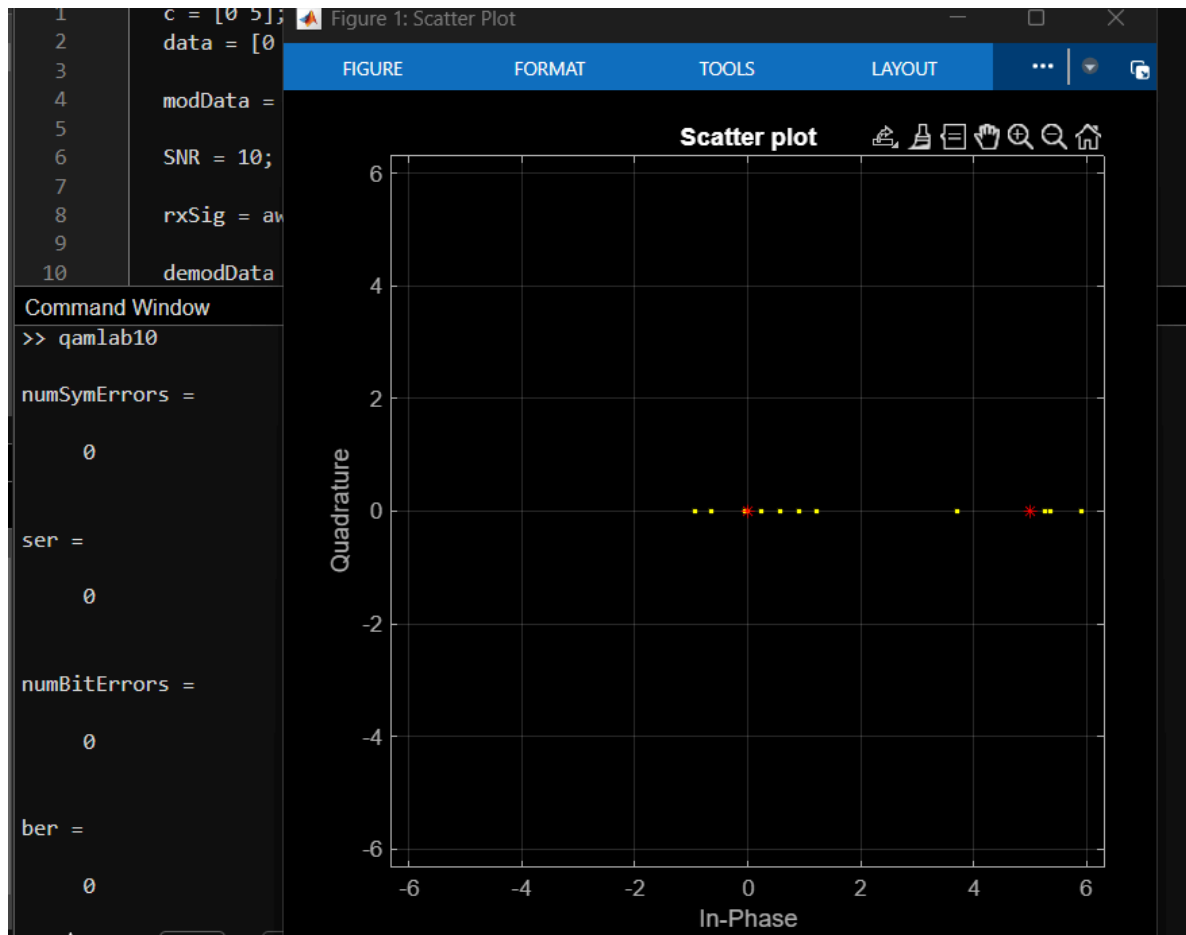
SNR = 10;

rxSig = awgn(modData,SNR,'measured');

demodData = genqamdemod(rxSig,c);

[numSymErrors,ser] = symerr(data,demodData)
[numBitErrors,ber] = biterr(data,demodData)

h = scatterplot(rxSig);
hold on
scatterplot(c,[],[],'r*',h)
grid
hold off
```



8. Examine the effect of decreasing/increasing the transmit power (i.e. decreasing/increasing transmit amplitude) on constellation shape and error rate. Try using a constellation of $c = [0 \ 3]$ to simulate reducing transmit power and then use $c = [0 \ 8]$ to simulate increasing it. Try this for $\text{SNR} = [3 \ 6 \ 10]$ again. Summarize the effect of decreasing or increasing transmit power on constellation shape and symbol/bit error rate, support your claims with screenshots.

Constellation of $c = [0 \ 3]$ and $\text{SNR} = 3$

```
c = [0 3];
data = [0 1 0 1 1 0 0 1 0 1 0 0 0];

modData = genqammod(data,c);

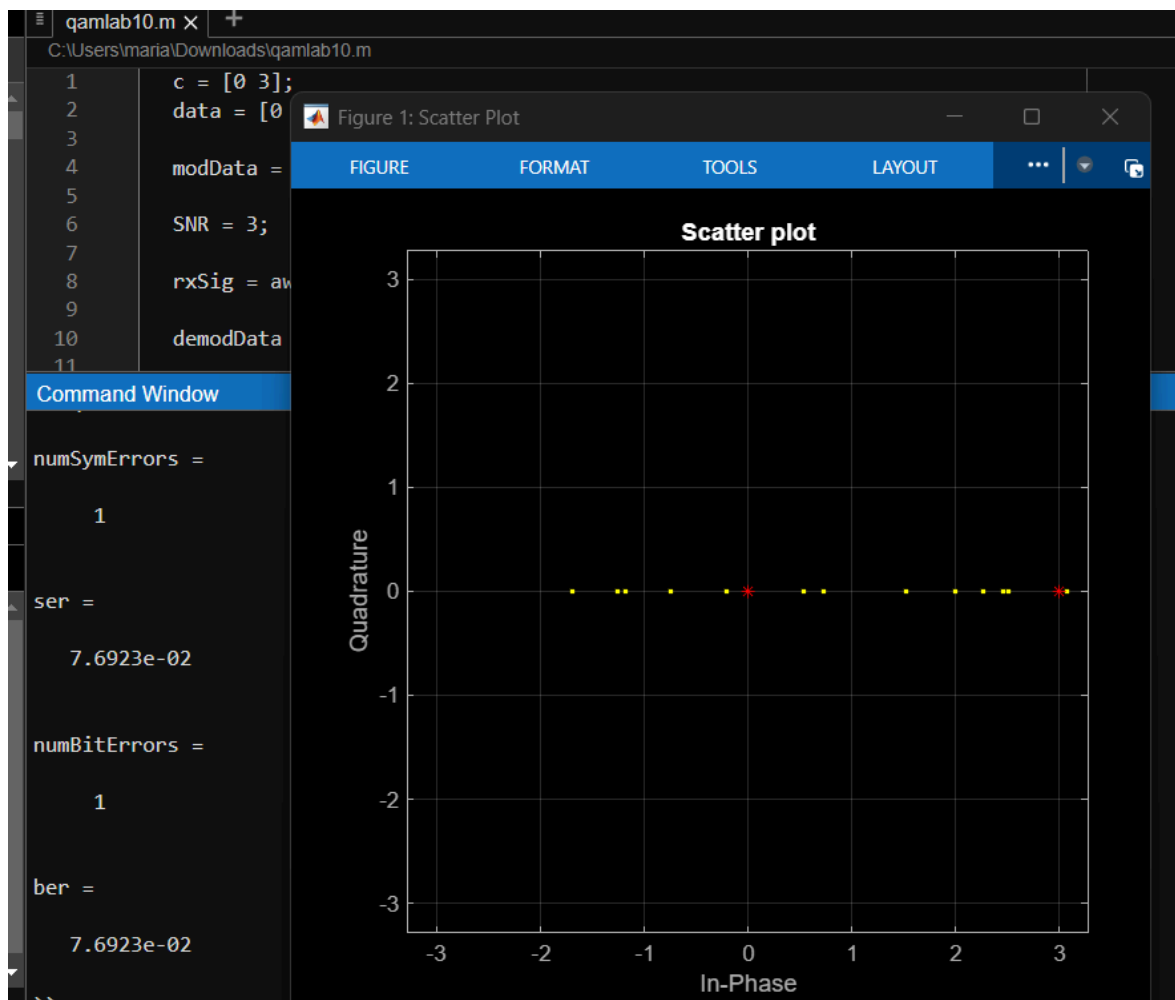
SNR = 3;

rxSig = awgn(modData,SNR,'measured');

demodData = genqamdemod(rxSig,c);

[numSymErrors,ser] = symerr(data,demodData)
[numBitErrors,ber] = biterr(data,demodData)

h = scatterplot(rxSig);
hold on
scatterplot(c,[],[],'r*',h)
grid
hold off
```



Constellation of $c = [0 \ 8]$ and $\text{SNR} = 3$

```

c = [0 8];
data = [0 1 0 1 1 0 0 1 0 1 0 0 0];

modData = genqammod(data,c);

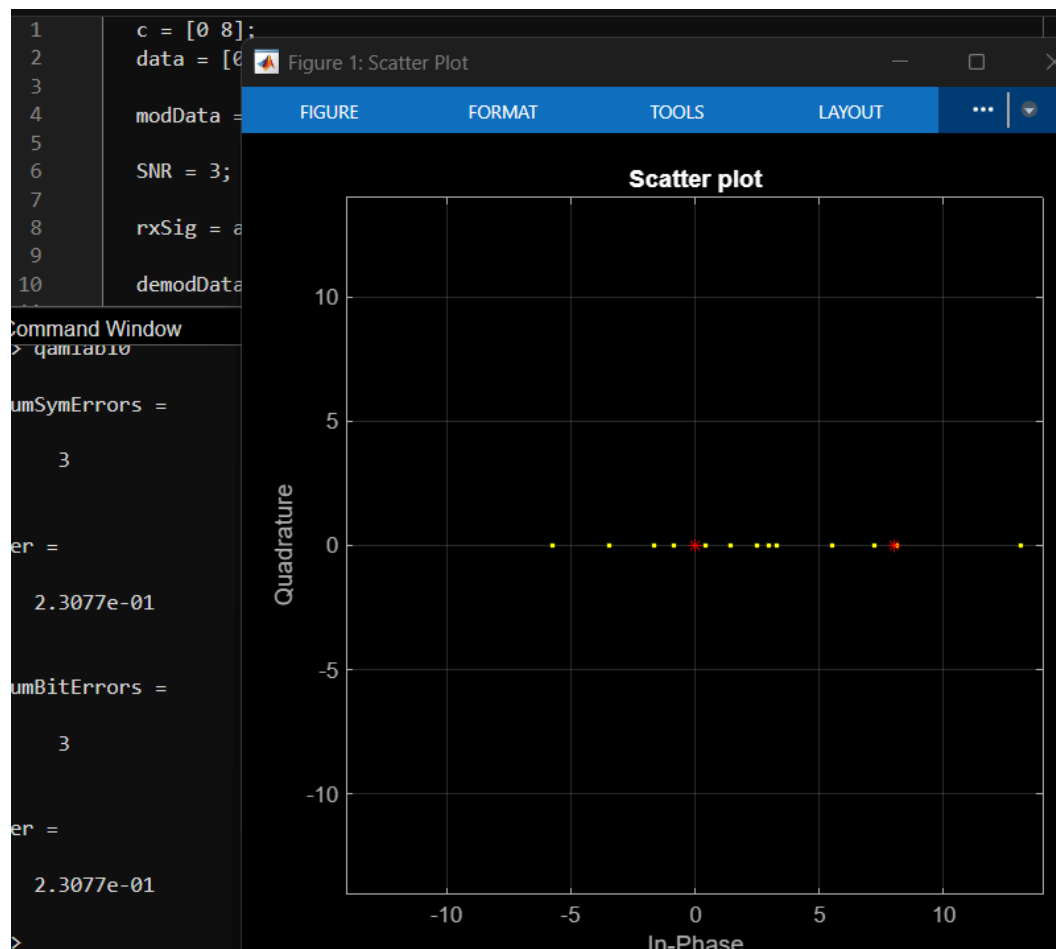
SNR = 3;

rxSig = awgn(modData,SNR,'measured');

demodData = genqamdemod(rxSig,c);

[numSymErrors,ser] = symerr(data,demodData)
[numBitErrors,ber] = biterr(data,demodData)

h = scatterplot(rxSig);
hold on
scatterplot(c,[],[],'r*',h)
grid
hold off
```



Constellation of $c = [0 \ 3]$ and $\text{SNR} = 6$

```
c = [0 3];
data = [0 1 0 1 1 0 0 1 0 1 0 0 0];

modData = genqammod(data,c);

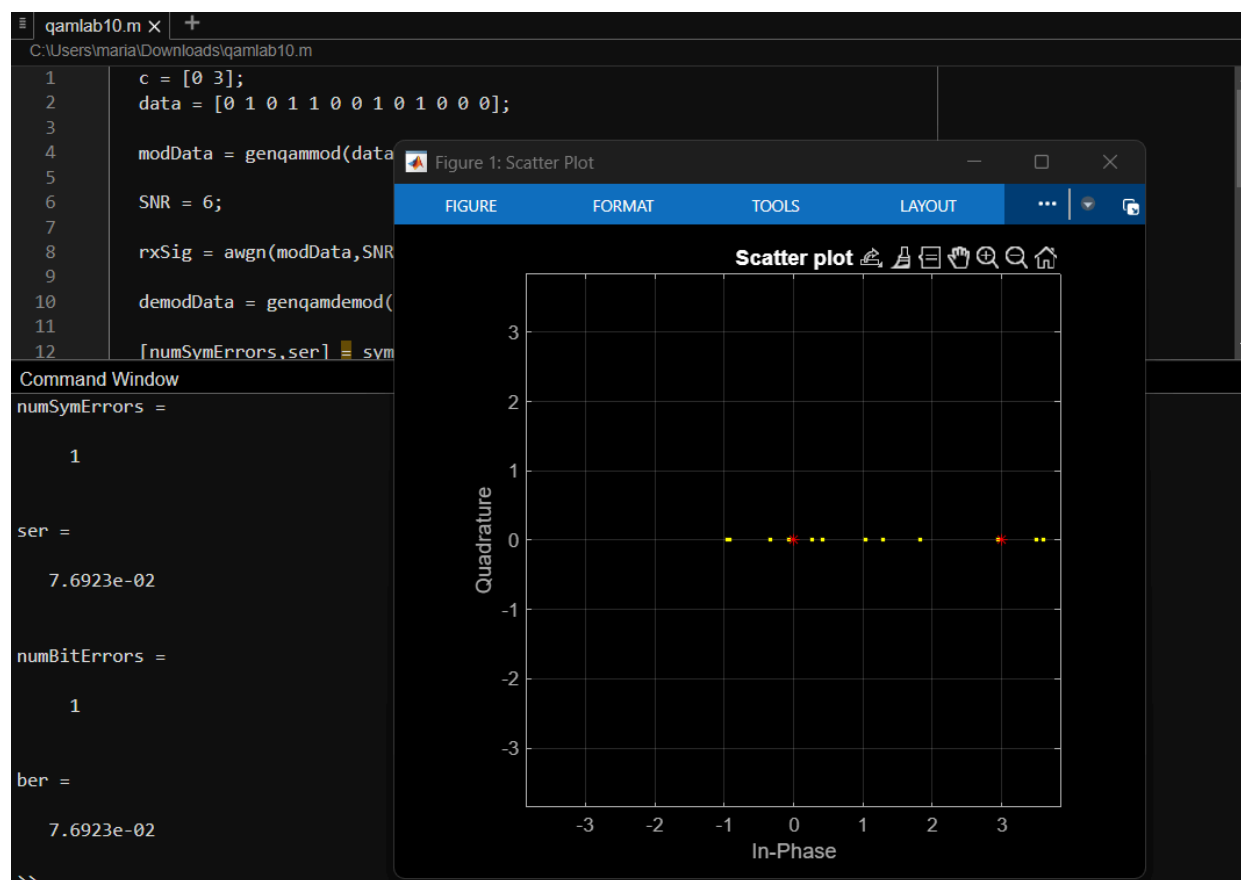
SNR = 6;

rxSig = awgn(modData,SNR,'measured');

demodData = genqamdemod(rxSig,c);

[numSymErrors,ser] = symerr(data,demodData)
[numBitErrors,ber] = biterr(data,demodData)

h = scatterplot(rxSig);
hold on
scatterplot(c,[],[],'r*',h)
grid
hold off
```



Constellation of $c = [0 \ 8]$ and $\text{SNR} = 6$

```
c = [0 8];
data = [0 1 0 1 1 0 0 1 0 1 0 0 0];

modData = genqammod(data,c);

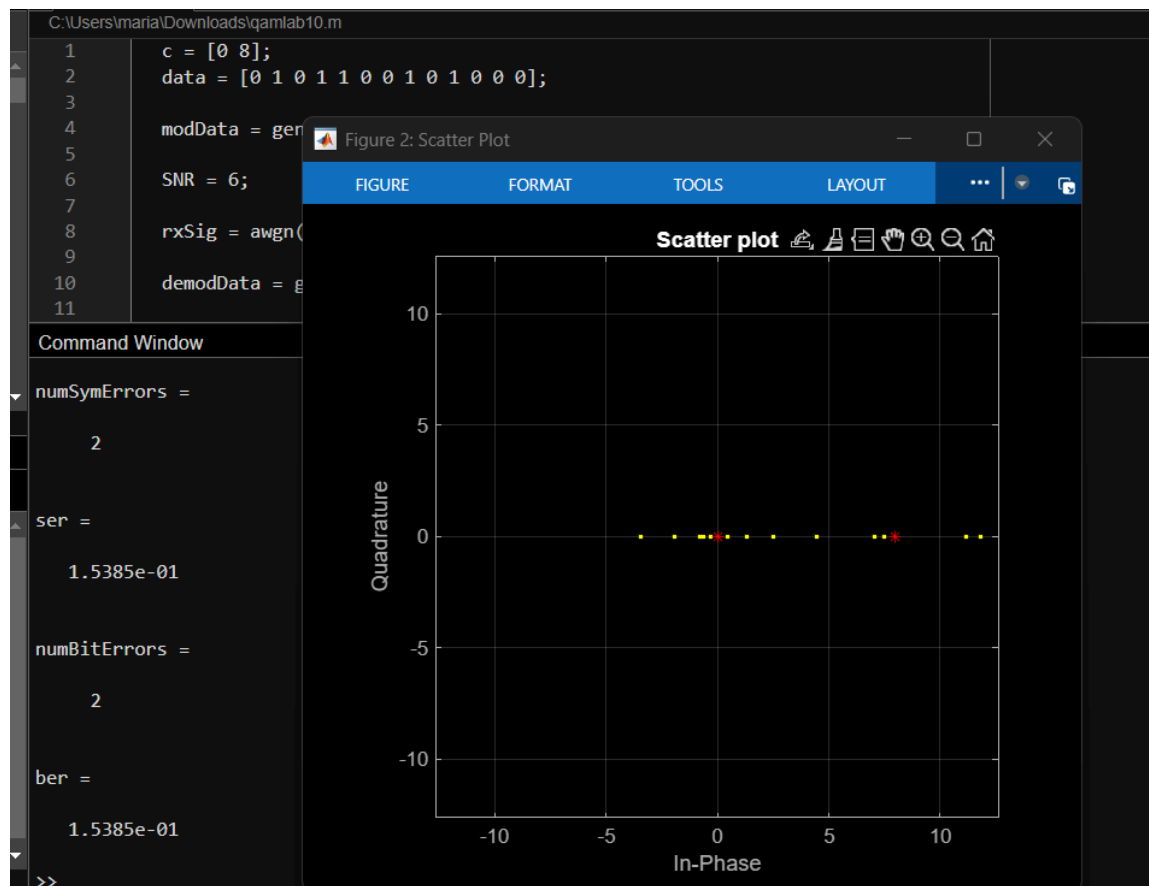
SNR = 6;

rxSig = awgn(modData,SNR,'measured');

demodData = genqamdemod(rxSig,c);

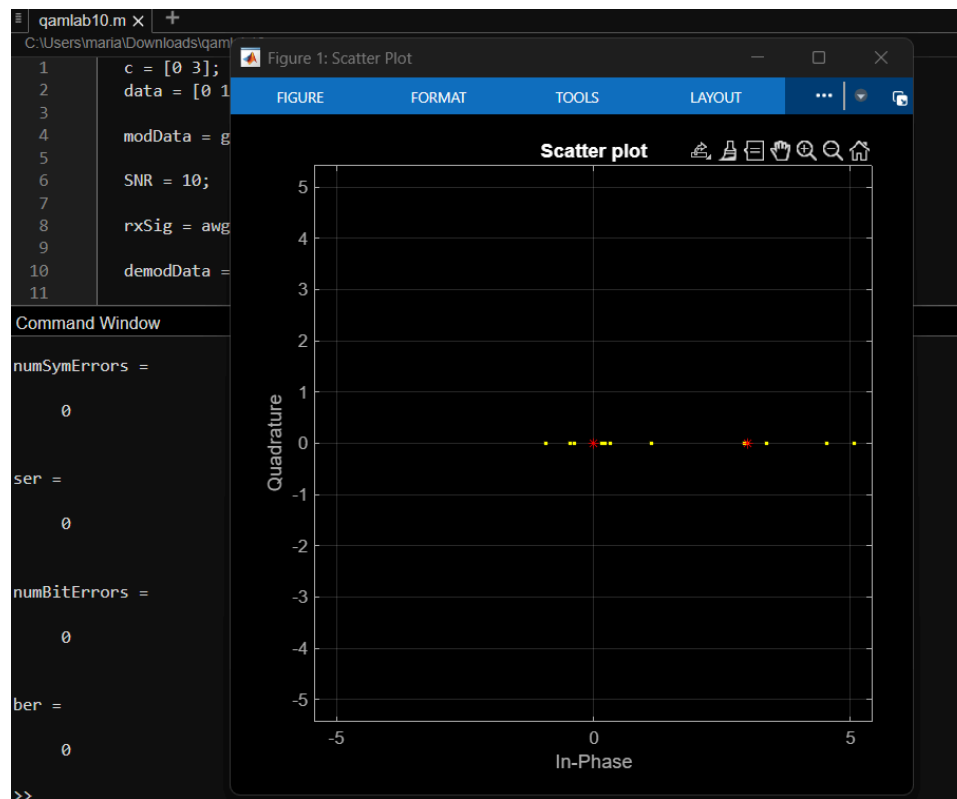
[numSymErrors,ser] = symerr(data,demodData)
[numBitErrors,ber] = biterr(data,demodData)

h = scatterplot(rxSig);
hold on
scatterplot(c,[],[],'r*',h)
grid
hold off
```



Constellation of $c = [0 \ 3]$ and $\text{SNR} = 10$

```
C:\Users\maria\Downloads\qamlab10.m
1  c = [0 3];
2  data = [0 1 0 1 1 0 0 1 0 1 0 0 0];
3
4  modData = genqammod(data,c);
5
6  SNR = 10;
7
8  rxSig = awgn(modData,SNR,'measured');
9
10 demodData = genqamdemod(rxSig,c);
11
12 [numSymErrors,ser] = symerr(data,demodData)
13 [numBitErrors,ber] = biterr(data,demodData)
14
15 h = scatterplot(rxSig);
16 hold on
17 scatterplot(c,[],[],'r*',h)
18 grid
19 hold off
```



Constellation of $c = [0 \ 8]$ and $\text{SNR} = 10$

```
J:\Users\maria\Downloads\qamlab10.m

c = [0 8];
data = [0 1 0 1 1 0 0 1 0 1 0 0 0];

modData = genqammod(data,c);

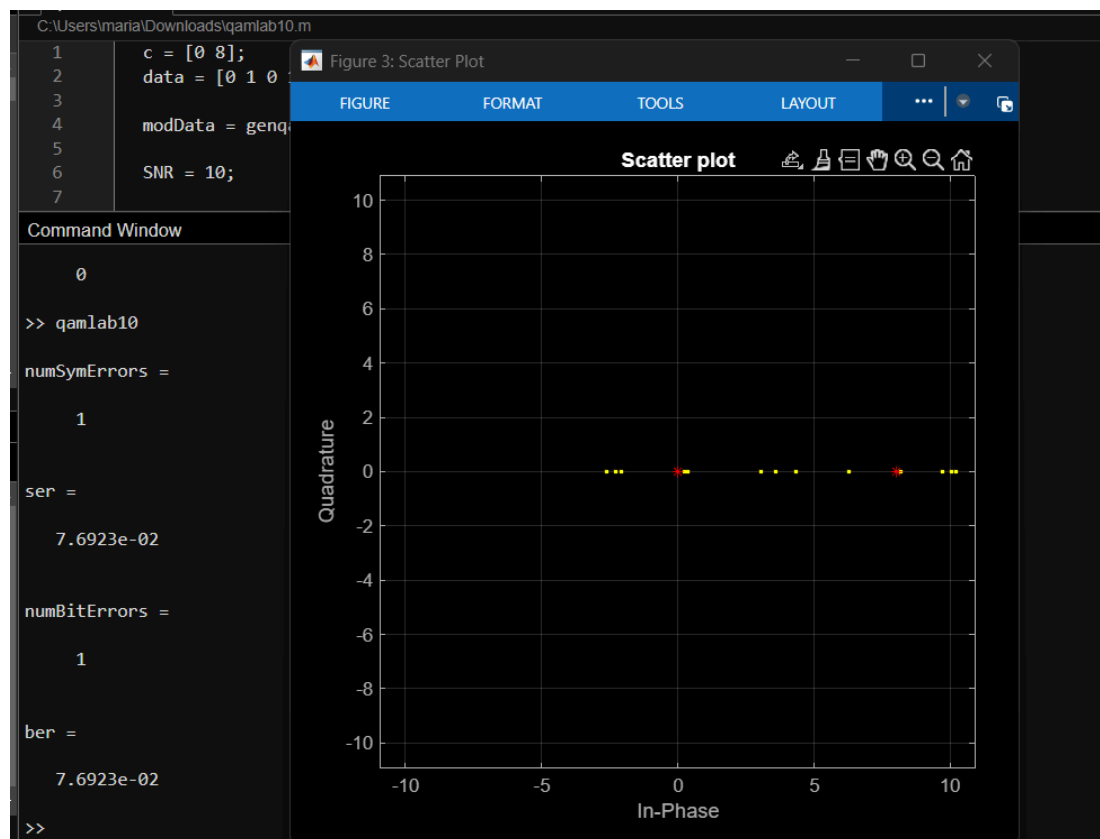
SNR = 10;

rxSig = awgn(modData,SNR,'measured');

demodData = genqamdemod(rxSig,c);

[numSymErrors,ser] = symerr(data,demodData)
[numBitErrors,ber] = biterr(data,demodData)

h = scatterplot(rxSig);
hold on
scatterplot(c,[],[],'r*',h)
grid
hold off
```



Summarize the effect of decreasing or increasing transmit power on constellation shape and symbol/bit/error rate.

As the transmit power increases, the constellation points are farther apart and thus have an increase in spacing. For lower transmit power like when $c = [0 \ 3]$, the constellation points are closer together making them more susceptible to noise and more chances for bit errors. Unlike when $c = [0 \ 8]$, the constellation points are spaced out further apart and reduces the chances for bit error. When $c = [0 \ 8]$, it is more reliable compared to when $c = [0 \ 3]$.

9. Compare the performance of 16-QAM and 16-ASK by creating one constellation vector for each.

```

1  clear;
2  clc;
3  close all;
4
5  M = 16;
6
7  c_qam = [-3-3i -3-1i -1-3i -1-1i ...
8           -3+3i -3+1i -1+3i -1+1i ...
9           3-3i 3-1i 1-3i 1-1i ...
10          3+3i 3+1i 1+3i 1+1i];
11
12  maxAmp = sqrt(2)*3;
13
14  stepSize = maxAmp/(M-1);
15  c_ask = 0:stepSize:maxAmp;
16
17  disp('16-QAM Constellation:');
18  disp(c_qam);
19
20  disp('16-ASK Constellation:');
21  disp(c_ask);

```

Command Window

```

16-QAM Constellation:
Columns 1 through 7
-3.0000 - 3.0000i -3.0000 - 1.0000i -1.0000 - 3.0000i -1.0000 - 1.0000i -3.0000 + 3.0000i -3.0000 + 1.0000i -1.0000 + 3.0000i
Columns 8 through 14
-1.0000 + 1.0000i 3.0000 - 3.0000i 3.0000 - 1.0000i 1.0000 - 3.0000i 1.0000 - 1.0000i 3.0000 + 3.0000i 3.0000 + 1.0000i
Columns 15 through 16
1.0000 + 3.0000i 1.0000 + 1.0000i
16-ASK Constellation:
Columns 1 through 15
0 0.2828 0.5657 0.8485 1.1314 1.4142 1.6971 1.9799 2.2627 2.5456 2.8284 3.1113 3.3941 3.6770 3.9598
Column 16
4.2426

```

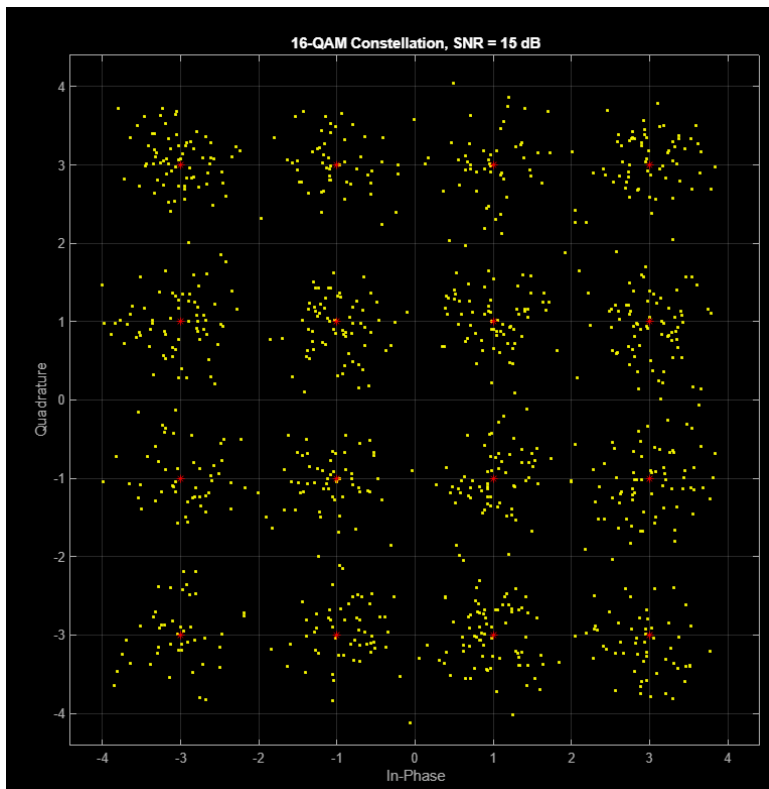
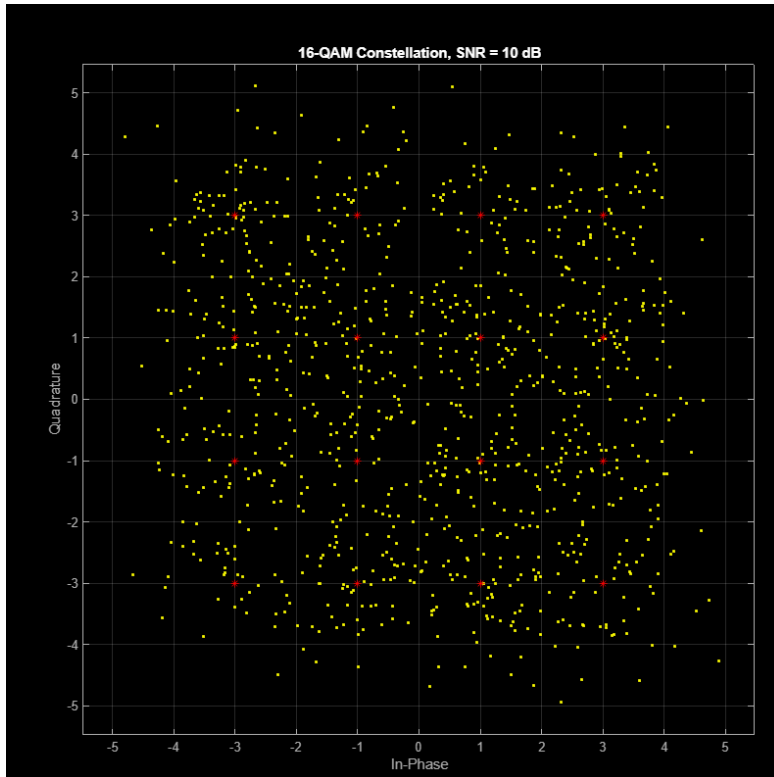
10 -11. Simplify the simulation and simultaneously make it more powerful, create a random sequence of 1000 data symbols to be sent. The data is a vector of random integers from 0 to M-1. Next, as before, modulate the same data onto the 16-QAM and the 16-ASK constellations (update your script to use “random_data” instead of “data”). And then add noise to the received signal.

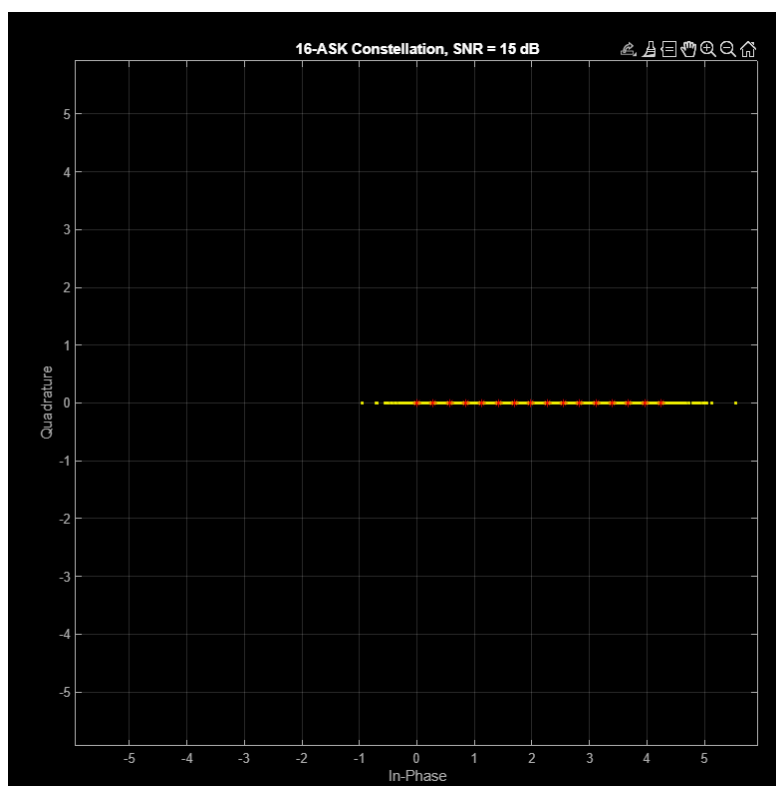
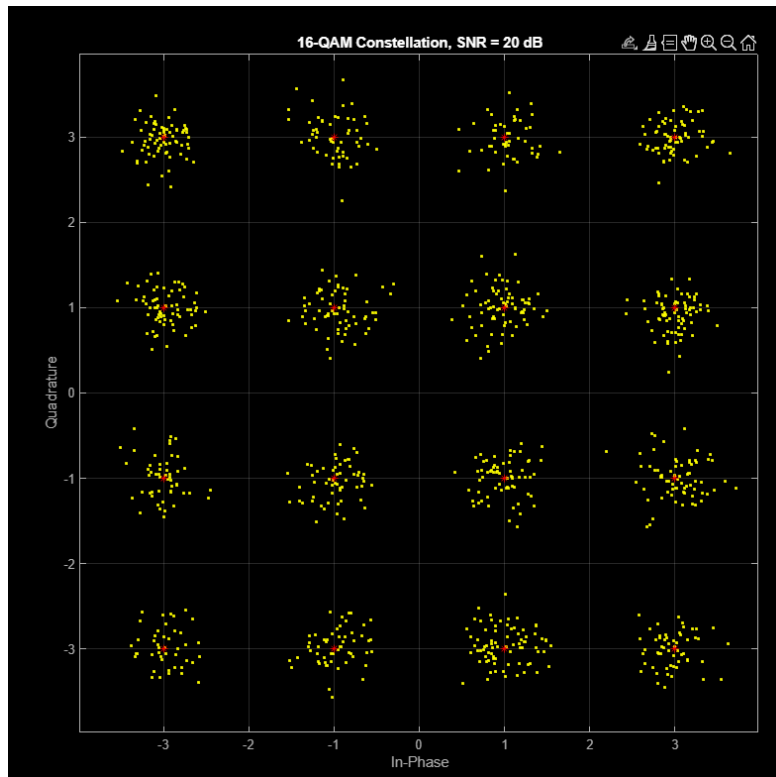
Experiment with different SNR values for 16-QAM and 16-ASK and compare. What is the minimum SNR needed to get the Symbol Error Rate (SER) below 10^{-2} (i.e. 1 error in 100 symbols or 1%) for each modulation scheme? Screen capture your constellation diagram plot for each. How about SER 10^{-3} (i.e. 1 error in 1000)?

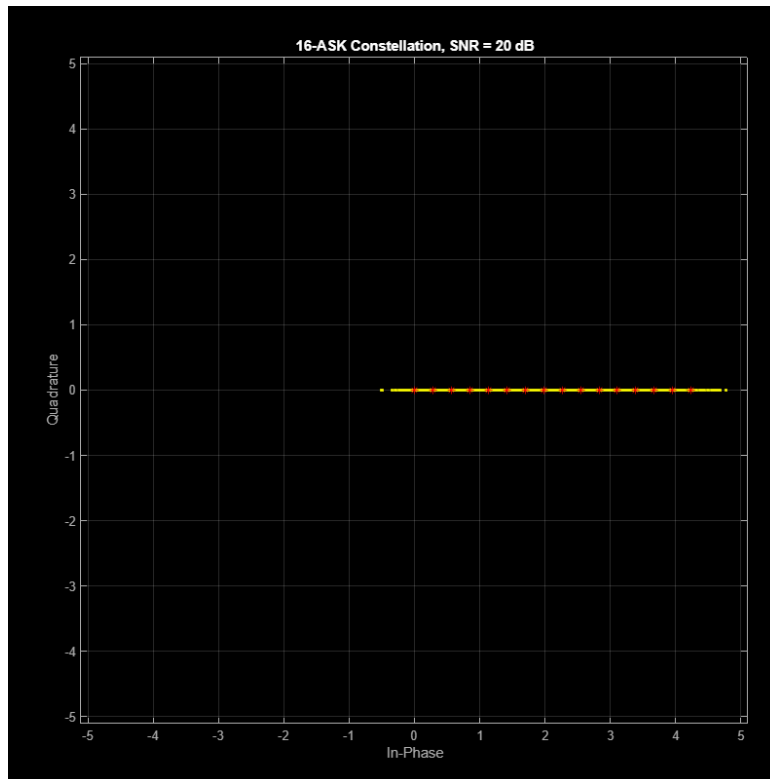
SNR(dB)	16-QAM SER	16-ASK SER
0	0.74100	0.86900
1	0.69700	0.88100
2	0.68900	0.89100
3	0.62900	0.88400
4	0.60300	0.88000
5	0.53900	0.86600
6	0.49000	0.86400
7	0.44300	0.84400
8	0.34300	0.83300
9	0.29300	0.79600
10	0.24000	0.80200
11	0.15900	0.80800
12	0.10600	0.75700
13	0.06100	0.75600
14	0.03500	0.72200
15	0.02400	0.69000
16	0.00700	0.67000
17	0.00300	0.67300
18	0.00200	0.59400
19	0.00000	0.56300
20	0.00000	0.52600
21	0.00000	0.48100
22	0.00000	0.44300
23	0.00000	0.40700
24	0.00000	0.34600
25	0.00000	0.28400
26	0.00000	0.24400
27	0.00000	0.19700
28	0.00000	0.15200
29	0.00000	0.09400
30	0.00000	0.06000

Minimum SNR for SER < 10^{-2} :
16-QAM: 16 dB
16-ASK: Not reached

Minimum SNR for SER < 10^{-3} :
16-QAM: 19 dB
16-ASK: Not reached







The minimum SNR needed to get the Symbol Error Rate (SER) below 10^{-2} is 16dB.

The minimum SNR needed to get the Symbol Error Rate (SER) below 10^{-3} is 19dB.

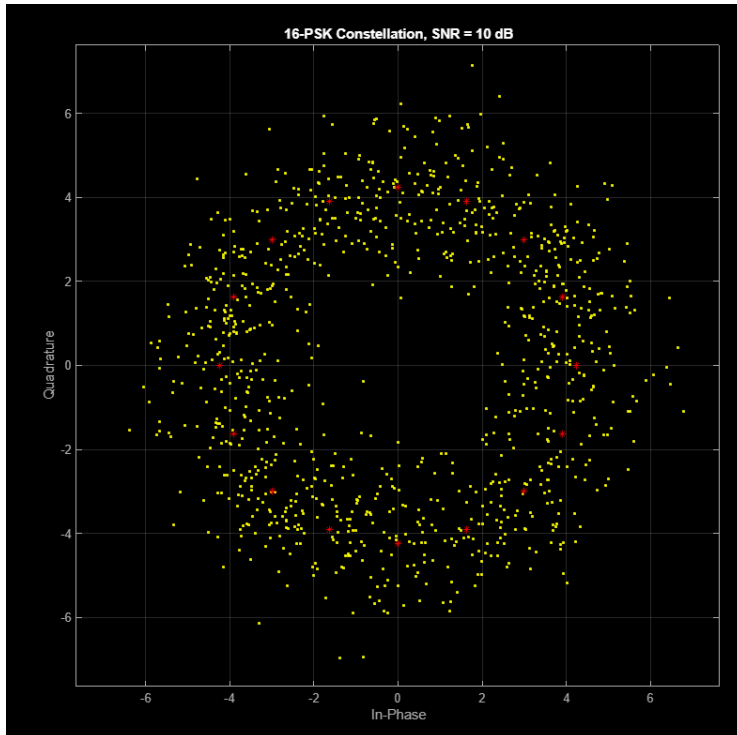
12. The distance between each symbol in the constellation will determine noise sensitivity. If the points are too close together, then small amounts of noise could cause a symbol to read incorrectly. What is the distance between points for the above 16 QAM constellation, and the 16-ASK constellation you created? Which is more sensitive to noise?

16-ASK is more sensitive to noise because its constellation points are much closer together and have less spacing than the 16-QAM points.

```
C:\Users\Gremiln\Downloads\labsstuff.m
1  clear;
2  clc;
3
4  M = 16;
5
6  c_qam = [-3-3i -3-1i -1-3i -1-1i ...
7           -3+3i -3+1i -1+3i -1+1i ...
8           3-3i 3-1i 1-3i 1-1i ...
9           3+3i 3+1i 1+3i 1+1i];
10
11  maxAmp = sqrt(2)*3;
12  stepSize = maxAmp/(M-1);
13  c_ask = 0:stepSize:maxAmp;
14
15  qam_points = [real(c_qam(:)) imag(c_qam(:))];
16
17
18  qam_distances = pdist(qam_points);
19  min_qam_distance = min(qam_distances);
20
21  ask_distances = diff(c_ask);
22  min_ask_distance = min(ask_distances);
23
24  fprintf('Minimum distance between 16-QAM points = %.4f\n', min_qam_distance);
25  fprintf('Minimum distance between 16-ASK points = %.4f\n', min_ask_distance);
26
27

Command Window
Minimum distance between 16-QAM points = 2.0000
Minimum distance between 16-ASK points = 0.2828
> Press Ctrl + Shift + P to generate code with Copilot
```

13. Create a 16-PSK constellation for comparison. Create a vector of 16 points that are all the same distance from the origin and spaced out evenly from each other. Compare this with the 16-ASK and 16-QAM.



16-QAM still gives the best performance out of the three because it uses both amplitude and phase dimensions which increases the spacing between the constellation points. 16-ASK is the worst because all 16 points are compressed along one axis. 16-PSK is better than 16-ASK since its points are spread around a circle but not better than 16-QAM.

TRANSMIT POWER, NOISE POWER, AND dBm

14. For the provided 16-QAM constellation, convert each constellation point's amplitude into power in Watts and in dBm. You can assume 50Ω for this analysis and that the voltages in the constellation diagram are peak values. You only need to do this for one quadrant (4 constellation points), since the other quadrants will have identical powers.

	Amplitude (Peak)	Amplitude (RMS)	Power (W) @ 50Ω	Power (dBm)
1+j	1.4142V	1.0000V	20mW	13dBm
1+3j	3.1623V	2.2361V	100mW	20dBm
3+j	3.1623V	2.2361V	100mW	20dBm
3+3j	4.2426V	3.0000V	180mW	22.dBm

```

C:\Users\Grenini\Downloads\labstuff.m
1  R = 50;
2  c_q1 = [1+1i 1+3i 3+1i 3+3i];
3
4  peak = abs(c_q1);
5  rms = peak / sqrt(2);
6  p_watts = rms.^2 / R;
7
8  fprintf('==== 16-QAM Power Table =====\n');
9  fprintf('Point\t\tPeak V\t\tRMS V\t\tPower W\t\tPower dBm\n');
10
11 for k = 1:length(c_q1)
12     fprintf('%s\t\t%.4f\t\t%.4f\t\t%.4f\t\t%.2f\n', ...
13         num2str(c_q1(k)), peak(k), rms(k), p_watts(k), p_dbm(k));
14 end
15
16 fprintf('\n');

```

Command Window

SNR = 10 dB | SER = 0.37400

```

>> labstuff
==== 16-QAM Power Table =====
Point      Peak V      RMS V      Power W      Power dBm
1+1i       1.4142      1.0000      0.0200      13.01
1+3i       3.1623      2.2361      0.1000      20.00
3+1i       3.1623      2.2361      0.1000      20.00
3+3i       4.2426      3.0000      0.1800      22.55

```

15. Assuming that each symbol in the quadrant is equally likely to be sent, what is the average power in both watts and dBm of the constellation? Hint: you just found the 4 possible power levels in the previous step. Average them using (linear) watts and then also convert that value to dBm.

Average constellation power: 100mW, 20dBm

16. The Signal to Noise ratio (SNR) is a measure of how large the average signal power is relative to the average noise power experienced by the channel and receiver. Remember that power is proportional to the square of the voltage, so you need to be careful about discussing power ratios or voltage ratios. Typically signal to noise ratio is measured in power and expressed in dB. For the following SNR's in dB, find the average noise power in watts and dB, and the average noise voltage in RMS and peak. Note you have already calculated Average Signal in a previous step.

SNR (dB)	SNR (linear)	Average Signal Power (W)	Average Noise Power (W)	Average Noise Power (dBm)	Average Noise Voltage (RMS)	Average Noise Voltage (Peak)
0	1.0000	100mW	0.100000	20dBm	2.2361	3.1623
6	3.9811	100mW	0.025119	14dBm	1.1207	1.5849
16	39.8107	100mW	0.002512	4dBm	0.3544	0.5012
30	1000.0000	100mW	0.000100	-10dBm	0.0707	0.1000

```

labstuff.m X
C:\Users\Gremim\Downloads\labstuff.m
1 SNR_db = [0 6 16 30];
2 SNR_linear = 10.^(SNR_db/10);
3
4 avg_signal_power = avg_power_watts;
5
6 avg_noise_power_watts = avg_signal_power ./ SNR_linear;
7 avg_noise_power_dbm = 10*log10(avg_noise_power_watts / 0.001);
8
9 avg_noise_voltage_rms = sqrt(avg_noise_power_watts * R);
10 avg_noise_voltage_peak = avg_noise_voltage_rms * sqrt(2);
11
12 fprintf('==== Noise Power Table =====\n');
13 fprintf('SNR dB\tSNR Linear\tSignal W\tNoise W\tNoise dBm\tNoise RMS V\tNoise Peak V\n');
14
15 for k = 1:length(SNR_db)
16     fprintf('%g\t%.4f\t%.4f\t%.6f\t%.2f\t%.4f\t%.4f\n', ...
17         SNR_db(k), SNR_linear(k), avg_signal_power, ...
18         avg_noise_power_watts(k), avg_noise_power_dbm(k), ...
19         avg_noise_voltage_rms(k), avg_noise_voltage_peak(k));
20 end

```

Command Window

Point	Peak V	RMS V	Power W	Power dBm
1+i	1.4142	1.0000	0.0200	13.01
1+3i	3.1623	2.2361	0.1000	20.00
3+i	3.1623	2.2361	0.1000	20.00
3+3i	4.2426	3.0000	0.1800	22.55

```

>> labstuff
===== Average 16-QAM Constellation Power =====
Average Power = 0.1000 W
Average Power = 20.00 dBm

>> labstuff
===== Noise Power Table =====
SNR dB SNR Linear Signal W Noise W Noise dBm Noise RMS V Noise Peak V
0 1.0000 0.1000 0.100000 20.00 2.2361 3.1623
6 3.9811 0.1000 0.025119 14.00 1.1207 1.5849
16 39.8107 0.1000 0.002512 4.00 0.3544 0.5012
30 1000.0000 0.1000 0.000100 -10.00 0.0707 0.1000

```

>> Press **Ctrl** + **Shift** + **P** to generate code with Copilot

QUESTIONS

1. Suppose I have a constellation $c2 = [1+i \ 1-i \ -1+i \ -1-i]$ and this constellation is mapped to some arbitrary data vector using `genqammod()`. We have 4 constellation points and this $M = 4$ symbols. The symbols are 0,1,2,3. Which symbol maps to which constellation point in $c2$?

Matlab maps the symbol numbers matching the order of the constellation vector. So the mapping is as follows:

Symbol	Constellation Point
0	1+i

1	$1-i$
2	$-1+i$
3	$-1-i$

2. What is the meaning of a SNR of 0 dB? How much higher in power in a linear scale is the signal compared to the noise at 0 dB? How about 3 dB, 6dB, 16 dB? This can be determined by taking the power ratio of signal over signal power in watts, from your table above.

The meaning of a SNR of 0 dB means the signal power and noise power are equal.

At 3 dB, the signal power is about 2x the noise power.

At 6 dB, the signal power is about 4x the noise power.

At 16dB, the signal power is about 40x the noise power.

CONCLUSION

This lab using MATLAB simulations showed that constellation spacing has a big effect on modulation performance. When SNR was increased, we saw the constellation points cluster more tightly around the ideal points which decreased the probability of symbol and bit errors. When we increased transmit amplitude, the constellation points grew further apart making it less sensitive to noise. When we compared 16-ASK, 16-QAM, and 16-PSK, we demonstrated that 16-QAM was the best performing and the least sensitive to noise. We did run into issues with syntax errors and figuring out the correct way to code in MATLAB. Overall, this lab reinforced our understanding of ASK, QAM, PSK, and how transmit power and SNR affects constellation point spacing.